

Tasks: Google Ads Integration (026)

Branch: 026-google-ads-integration

Input: Design documents from `specs/026-google-ads-integration/`

Prerequisites: plan.md ✓ | spec.md ✓ | research.md ✓ | data-model.md ✓ | contracts/ ✓ | quickstart.md ✓

Organization: Tasks grouped by user story — each phase is independently deliverable and testable.

Format: [ID] [P?] [Story] Description

- **[P]:** Can run in parallel (different files, no shared dependencies)
 - **[Story]:** Maps to user stories from spec.md (US1–US4)
 - Exact file paths included in every task description
-

Phase 1: Setup (Shared Infrastructure)

Purpose: Configure environment variables and inject the AdSense script — prerequisites for ALL ad unit work.

- ✓ T001 Add `VITE_ADSENSE_PUBLISHER_ID`, `VITE_ADSENSE_SLOT_GATE`, `VITE_ADSENSE_SLOT_DISCOVER`, `VITE_ADSENSE_SLOT_TRACK`, `VITE_ADSENSE_SLOT_ARTIST` to `client/.env` (use placeholder values for local dev)
- ✓ T002 [P] Document the five new `VITE_ADSENSE_*` variables with comments in `client/.env.example`
- ✓ T003 [P] Inject the AdSense async `<script>` tag using `%VITE_ADSENSE_PUBLISHER_ID%` Vite interpolation in the `<head>` of `client/index.html` (placed after the existing dark-mode script block)

Checkpoint: AdSense script loads in browser when `VITE_ADSENSE_PUBLISHER_ID` is set. Verify via DevTools → Network filter `adsbygoogle`.

Phase 2: Foundational (Blocking Prerequisites)

Purpose: Backend config extension and the core `GoogleAdUnit` React component — required before any ad placement or admin UI work can begin.

- ✓ T004 Extend `AdsConfig` type in `api/src/common/ads-config.util.ts`: add `googleAdsEnabled: boolean` field, raise `gateSeconds` default from 5 to 30, update `mergeAdsConfig()` to include the new field
 - ✓ T005 [P] Add `@IsOptional()` `@IsBoolean()` `googleAdsEnabled?: boolean` field to `api/src/modules/ads/dto/update-ads-config.dto.ts`
 - ✓ T006 Update `ads.service.ts` at `api/src/modules/ads/ads.service.ts`: persist `googleAdsEnabled` in `updateConfig()` and return it from `getPublicConfig()` /
-

getConfig()

- ☒ T007 [P] Extend the `AdsConfig` TypeScript type in `client/src/hooks/useAds.ts` to include `googleAdsEnabled: boolean`
- ☒ T008 Create `client/src/components/ads/GoogleAdUnit.tsx` — a reusable `<ins class="adsbygoogle">` wrapper that: reads `VITE_ADSENSE_PUBLISHER_ID` from `import.meta.env` and returns `null` if absent; accepts props `slotId: string`, `format: 'auto' | 'rectangle' | 'leaderboard'`, `className?: string`; calls `(window.adsbygoogle = window.adsbygoogle || []).push({})` in `useEffect`; wraps the push in `try/catch` to silently swallow errors when ad-blockers are active; sets correct `data-ad-client` and `data-ad-slot` attributes on the `<ins>` element

Checkpoint: `npm --prefix api run build` passes. `GoogleAdUnit` renders an `<ins>` tag in the DOM when publisher ID is set, and renders nothing (`null`) when absent.

Phase 3: User Story 1 — Ad-Gate Google Ad (Priority: P1) 🎯 MVP

Goal: Display a real Google AdSense advertisement inside the ad-gate countdown modal before a fan downloads a free track.

Independent Test: Navigate to any free track → click Download → confirm `<ins class="adsbygoogle" data-ad-slot="GATE_SLOT">` is present inside the countdown modal card; confirm countdown still reaches zero and download unlocks regardless of whether an ad fills the slot.

Implementation for User Story 1

- ☒ T009 [US1] Modify `client/src/components/ads/AdGateModal.tsx`: import `GoogleAdUnit` and `useAdsConfig`; replace the `<img>` custom creative block with a conditional render — when `googleAdsEnabled` is `true` and `VITE_ADSENSE_SLOT_GATE` is set, render `<GoogleAdUnit slotId={slotGate} format="rectangle" />`; keep the existing `<img>` creative as the `else` fallback branch
- ☒ T010 [US1] Pass `googleAdsEnabled` (from `useAdsConfig()`) and `slotGate` (`import.meta.env.VITE_ADSENSE_SLOT_GATE`) into `AdGateModal` via props or read them directly inside the component — choose whichever keeps the component interface clean; update the `AdGateModalProps` type in `client/src/components/ads/AdGateModal.tsx` accordingly

Checkpoint (US1): With `VITE_ADSENSE_PUBLISHER_ID` and `VITE_ADSENSE_SLOT_GATE` set → open ad gate on a free track → `<ins>` element visible in modal → countdown completes → download works. With an ad-blocker enabled → gate still opens → countdown completes → download works → zero console errors.

Phase 4: User Story 2 — Ambient Display Ads (Priority: P2)

Goal: Display Google AdSense leaderboard and rectangle ad units on the Discover, Track, and Artist Profile pages.

Independent Test: Navigate to each of the three pages and confirm the `<ins class="adsbygoogle">` element is present at the correct position in the DOM. Confirm no horizontal overflow at 375px viewport width.

Implementation for User Story 2

-  T011 [P] [US2] Modify `client/src/pages/DiscoverPage.tsx`: import `GoogleAdUnit` and render `<GoogleAdUnit slotId={import.meta.env.VITE_ADSENSE_SLOT_DISCOVER} format="leaderboard" className="my-4" />` between the hero/header section and the music grid; wrap in a conditional so it only renders when `googleAdsEnabled` is `true` (read from `useAdsConfig()`)
-  T012 [P] [US2] Modify `client/src/pages/TrackPage.tsx`: import `GoogleAdUnit` and render `<GoogleAdUnit slotId={import.meta.env.VITE_ADSENSE_SLOT_TRACK} format="rectangle" className="mt-6" />` below the download/stream action button area; guard with `googleAdsEnabled` from `useAdsConfig()`
-  T013 [P] [US2] Modify `client/src/pages/ArtistProfilePage.tsx`: import `GoogleAdUnit` and render `<GoogleAdUnit slotId={import.meta.env.VITE_ADSENSE_SLOT_ARTIST} format="leaderboard" className="my-4" />` between the artist header section and the track list; guard with `googleAdsEnabled` from `useAdsConfig()`

Checkpoint (US2): All three pages show the `<ins>` elements in the correct positions at desktop viewport. At 375px width, no horizontal scroll. When `googleAdsEnabled` is toggled `false` via the admin panel, reloading any page shows no `<ins>` elements.

Phase 5: User Story 3 — Admin Configuration Panel (Priority: P2)

Goal: Allow the platform admin to view AdSense configuration status and toggle Google Ads on/off from the admin panel.

Independent Test: Log in as admin → navigate to Admin → Ads → confirm a "Google AdSense" card section is visible showing publisher configured/not configured status and a `googleAdsEnabled` toggle. Toggle off → navigate to Discover → confirm no `<ins>` elements. Toggle back on → confirm elements return.

Implementation for User Story 3

-  T014 [US3] Modify `client/src/pages/AdminAdsPage.tsx`: add a new "Google AdSense" `<Card>` section below the existing ad settings card; the card must: read `VITE_ADSENSE_PUBLISHER_ID` from `import.meta.env` and display a status badge (" Publisher configured" or " Not configured"); display the current `googleAdsEnabled` value from `useAdminAdsConfig()` data; render a toggle button that calls `updateConfig.mutateAsync({ googleAdsEnabled: !current })` using the existing `useUpdateAdsConfig` hook; when publisher ID is absent, display a short setup guide: "Set `VITE_ADSENSE_PUBLISHER_ID` in `client/.env` and restart the dev server"

Checkpoint (US3): Admin toggle changes `googleAdsEnabled` via `PUT`

`/api/v1/admin/ads/config`. The change is reflected immediately in the admin UI and takes effect on the next fan-facing page navigation.

Phase 6: User Story 4 — Artist Ad-Supported Label (Priority: P3)

Goal: Display an "Ad-supported" label on free tracks in the artist dashboard, building transparency with artists.

Independent Test: Log in as an artist with at least one free track → navigate to artist dashboard track list → confirm "Ad-supported" badge is visible on the free track row → hover/tap the badge → confirm tooltip text appears.

Implementation for User Story 4

- ☒ T015 [US4] Modify `client/src/pages/ArtistDashboardPage.tsx`: in the track list rendering section, add a conditional badge for tracks where `pricingType === 'FREE'`; the badge should read "Ad-supported" and include a tooltip (using the existing `shadcn/ui` `Tooltip` or a `title` attribute) with text: "This track is free for fans. Ad revenue helps sustain the platform."

Checkpoint (US4): Every free track row in the artist dashboard shows the "Ad-supported" label. Paid and pay-what-you-want tracks show no label.

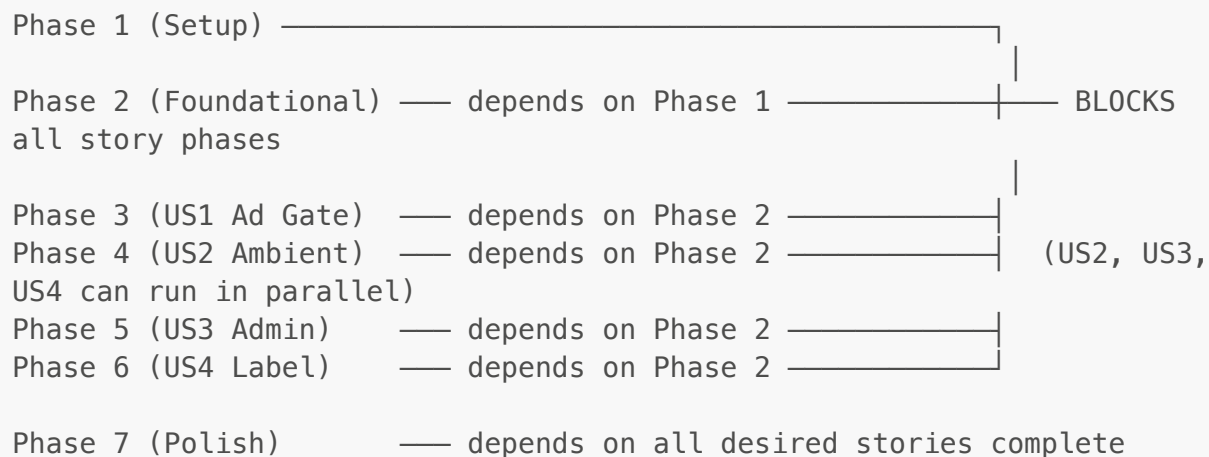
Phase 7: Polish & Cross-Cutting Concerns

Purpose: Auto ads enablement, build validation, and layout verification.

- ☒ T016 [P] Enable Google Auto ads by confirming the AdSense `<script>` tag in `client/index.html` has `data-ad-client` attribute set via `%VITE_ADSENSE_PUBLISHER_ID%` — Auto ads are activated automatically by the AdSense script when the publisher account has Auto ads enabled in the AdSense dashboard (no additional code change required; document this in `client/.env.example`)
 - ☒ T017 [P] Verify all `GoogleAdUnit` usages have `max-w-full` or equivalent Tailwind class on their container to prevent horizontal overflow at narrow viewports; check `client/src/pages/DiscoverPage.tsx`, `TrackPage.tsx`, `ArtistProfilePage.tsx`, and `client/src/components/ads/AdGateModal.tsx`
 - ☒ T018 [P] Run `npm --prefix api run build` and confirm zero TypeScript errors
 - ☒ T019 [P] Run `npm --prefix client run build` and confirm zero TypeScript/Vite build errors
 - ☒ T020 Run the full validation checklist from `specs/026-google-ads-integration/quickstart.md` steps 1–7 and mark each step pass/fail
-

Dependencies & Execution Order

Phase Dependencies



Within Phase 2

- T004 → T006 (service depends on updated AdsConfig type)
- T005, T007, T008 can run in parallel with T004

Within Phase 3 (US1)

- T008 (GoogleAdUnit component) must be complete before T009
- T009 → T010 (props type update follows component integration)

Parallel Opportunities per Story

Story	Parallel Tasks
US2	T011, T012, T013 — all different files, no shared dependencies
US3	T014 — standalone admin page modification
US4	T015 — standalone dashboard page modification
Polish	T016, T017, T018, T019 — all independent

Implementation Strategy

MVP (User Story 1 Only — Ad Gate)

1. Complete **Phase 1**: Set env vars, inject AdSense script → `client/index.html`
2. Complete **Phase 2**: Extend `AdsConfig`, create `GoogleAdUnit.tsx` → API + client foundation
3. Complete **Phase 3**: Integrate `GoogleAdUnit` into `AdGateModal.tsx`
4. **Validate**: Follow quickstart.md steps 1–4 → ad renders in gate → countdown works → download unlocks
5. **Deploy/Demo** — the single most revenue-impactful placement is live

Incremental Delivery

1. MVP (Phase 1–3) → Ad gate monetised ✅

2. Add Phase 4 → Ambient display ads on 3 pages ✓
 3. Add Phase 5 → Admin can toggle Google Ads globally ✓
 4. Add Phase 6 → Artist transparency labels ✓
 5. Phase 7 → Build clean, layout verified, quickstart validated ✓
-

Notes

- No new NestJS module, no TypeORM migration — this feature is purely additive
- `GoogleAdUnit` must never throw or log errors — all AdSense failures must be silent (try/catch + null return)
- `googleAdsEnabled` guard must wrap every `<GoogleAdUnit>` render call across all pages
- Auto ads (FR-012) require no code beyond the `<script>` tag — Google handles the rest
- All API routes already use `/api/v1/` prefix — no route changes needed
- Client env vars prefixed `VITE_ADSENSE_*` — all optional; absent = unit renders null